

Sistema CATMAT

Este diretório reúne os três serviços do sistema:

- `mongodb` : banco de dados com a carga CATMAT.
- `back` : API NestJS de classificação e geração dos arquivos.
- `front` : interface React/Vite para upload da planilha e download dos resultados.

Componentes Do Sistema

Frontend

O frontend fica em:

```
front/
```

Ele é uma aplicação React com Vite. A função dele é oferecer a interface para o usuário:

- baixar o modelo de planilha de entrada;
- selecionar ou arrastar a planilha patrimonial;
- validar apenas extensão e tamanho do arquivo;
- pedir confirmação antes do envio;
- enviar o arquivo para o backend no endpoint de importação;
- mostrar o resumo do processamento;
- baixar a planilha de saída e a planilha de auditoria.

O frontend não interpreta nem classifica a planilha. Ele só envia o arquivo para a API e exibe o resultado retornado. A URL da API é definida por:

```
VITE_API_BASE_URL=http://localhost:3000
```

No Docker Compose, esse valor é configurado automaticamente para apontar para a porta local do backend.

Backend

O backend fica em:

```
back/
```

Ele é uma API NestJS responsável por todo o processamento da classificação CATMAT:

- receber upload de planilhas `.csv`, `.xlsx` ou `.xls`;
- validar se o arquivo foi enviado corretamente;

- validar tipo e tamanho do arquivo;
- ler a planilha e conferir as colunas obrigatórias;
- extrair a descrição patrimonial de cada linha;
- buscar candidatos CATMAT no MongoDB;
- ranquear candidatos encontrados;
- chamar a LLM quando necessário;
- decidir o CATMAT selecionado;
- gerar a planilha final de saída;
- gerar a planilha de auditoria;
- expor links para download dos arquivos gerados.

Endpoints principais:

```
POST /catmat-classificacao/importar
GET /catmat-classificacao/download/:fileName
```

Os arquivos gerados ficam em:

```
back/exports/
```

MongoDB

O MongoDB armazena a base CATMAT usada pelo backend para buscar candidatos de classificação.

Durante a inicialização, o container importa a carga oficial para a coleção configurada por:

```
MONGO_CATMAT_COLLECTION=catmat
```

Depois da importação, o script de inicialização cria índice textual nos campos:

```
nomeGrupo
nomeClasse
descricaoItem
```

Esse índice permite que o backend faça busca textual por candidatos CATMAT a partir da descrição do bem patrimonial enviada na planilha.

Script De Coleta CATMAT

O script fica em:

```
back/scripts/catmat/fetch-catmat.py
```

Ele consulta a API pública de dados abertos do Compras.gov.br:

```
https://dadosabertos.compras.gov.br/modulo-material/4_consultarItemMaterial
```

O script percorre as páginas da API, junta os itens retornados e gera um JSON consolidado com o campo `resultado`. Esse arquivo consolidado é a base usada para montar a carga CATMAT do MongoDB.

Por padrão, o script gera:

```
itens-material-completo.json
```

Depois de gerar ou atualizar esse arquivo, copie/renomeie o conteúdo final para:

```
back/data/catmat/carga.json
```

Arquivo `carga.json`

O arquivo fica em:

```
back/data/catmat/carga.json
```

Ele contém a base CATMAT que será importada para o MongoDB. O formato esperado é um objeto JSON com o array `resultado`:

```
{
  "totalRegistros": 342660,
  "totalPaginas": 686,
  "totalItensColetados": 342660,
  "dataGeracao": "2026-06-18T00:00:00",
  "resultado": [
    {
      "codigoItem": 206504,
      "nomeGrupo": "MOBILIÁRIOS",
      "nomeClasse": "MOBILIÁRIO PARA ESCRITÓRIO",
      "descricaoItem": "CADEIRA ESCRITÓRIO...",
      "statusItem": true
    }
  ]
}
```

Na primeira subida do MongoDB, o arquivo é montado no container em:

```
/carga/carga.json
```

e importado pelo script:

```
back/docker/mongo/init/01-load-catmat.js
```

Se o volume `mongodb_data` já existir, a carga não será refeita automaticamente. Para importar novamente, remova o volume com `docker compose down -v`.

Pré-requisitos

Para subir tudo com Docker Compose:

```
Docker  
Docker Compose
```

Instalando Docker E Docker Compose

O caminho recomendado depende do seu sistema operacional:

- Windows ou macOS: instale o Docker Desktop pela documentação oficial: <https://docs.docker.com/get-started/get-docker/>
- Linux com interface gráfica: você também pode usar Docker Desktop para Linux: <https://docs.docker.com/get-started/get-docker/>
- Linux servidor/terminal: instale o Docker Engine pela documentação oficial da sua distribuição: <https://docs.docker.com/engine/install/>

O Docker Compose atual é usado pelo comando:

```
docker compose
```

Em instalações com Docker Desktop, o Docker Compose normalmente já vem incluído. Em instalações com Docker Engine no Linux, instale o plugin do Compose seguindo a documentação oficial:

```
https://docs.docker.com/compose/install/
```

Depois de instalar, confirme se os comandos estão disponíveis:

```
docker --version  
docker compose version
```

Para subir os serviços separadamente:

```
Node.js 22 ou compatível
npm
Docker ou MongoDB local
```

Configuração

O Compose usa valores padrão suficientes para ambiente local. Se quiser alterar portas ou parâmetros CATMAT, crie um arquivo `.env` neste diretório:

```
cd sistema
cp .env.example .env
```

Principais variáveis:

```
MONGO_PORT=27017
BACK_PORT=3000
FRONT_PORT=5173
OPENAI_API_KEY=
```

Configurando A `OPENAI_API_KEY`

A `OPENAI_API_KEY` é necessária quando o backend precisar chamar a LLM para decidir a classificação CATMAT. Sem ela, o sistema ainda sobe, mas as linhas que dependerem da LLM podem falhar no processamento.

Para usar com o Docker Compose principal, configure a chave no `.env` do backend:

```
cd sistema/back
cp .env.example .env
```

Abra o arquivo `sistema/back/.env` e preencha:

```
OPENAI_API_KEY=sua_chave_aqui
```

Depois recrie o container do backend para ele carregar a variável:

```
cd sistema
docker compose up -d --force-recreate back
```

Para conferir se a chave foi carregada sem exibir o valor:

```
docker compose exec back node -e 'console.log("OPENAI_API_KEY carregada:",
    Boolean(process.env.OPENAI_API_KEY), "tamanho:",
    process.env.OPENAI_API_KEY?.length || 0)'
```

No Docker Compose principal, o serviço `back` carrega o arquivo `sistema/back/.env`. As variáveis específicas do ambiente Docker, como `MONGO_URI=mongodb://mongodb:27017`, continuam definidas no `docker-compose.yml` para apontar para o serviço MongoDB interno.

Carga CATMAT

O MongoDB é inicializado com a carga em:

```
back/data/catmat/carga.json
```

Na primeira criação do volume `mongodb_data`, o container do Mongo importa `carga.json` e cria o índice textual usado pela busca CATMAT.

Se modificar a carga e quiser recriar o banco do zero:

```
cd sistema
docker compose down -v
docker compose up
```

Subindo Tudo Com Docker Compose

Na raiz do diretório `sistema`, execute:

```
docker compose up
```

Para rodar em segundo plano:

```
docker compose up -d
```

Endereços:

```
Front: http://localhost:5173
Back:  http://localhost:3000
Mongo: localhost:27017
```

O Compose faz:

- sobe o MongoDB;
- aguarda o healthcheck do Mongo;

- instala dependências do backend com `npm ci` ;
- aguarda o backend conseguir resolver e conectar em `mongodb:27017` ;
- inicia o backend com `npm run start:dev` ;
- instala dependências do frontend com `npm ci` ;
- inicia o frontend com `npm run dev -- --host 0.0.0.0 --port 5173` .

Comandos Úteis Do Compose

Ver logs de todos os serviços:

```
docker compose logs -f
```

Ver logs apenas do backend:

```
docker compose logs -f back
```

Ver logs apenas do frontend:

```
docker compose logs -f front
```

Parar os serviços sem apagar volumes:

```
docker compose down
```

Parar e apagar volumes, incluindo a base Mongo:

```
docker compose down -v
```

Recriar containers:

```
docker compose up --force-recreate
```

Se aparecer `getaddrinfo EAI_AGAIN mongodb` , pare e recrie o compose principal:

```
cd sistema  
docker compose down  
docker compose up --force-recreate
```

Se antes você subiu o Mongo pelo compose isolado do backend, pare aquele stack também:

```
cd sistema/back
docker compose down
```

Validar a configuração renderizada:

```
docker compose config
```

Subindo Separadamente

Use este fluxo se quiser rodar backend e frontend direto na máquina, deixando só o Mongo no Docker, ou se preferir depurar cada serviço isoladamente.

1. Subir o MongoDB

A opção mais simples é usar o compose específico do backend:

```
cd sistema/back
docker compose up -d mongodb
```

Esse compose monta:

```
sistema/back/data/catmat/carga.json -> /carga/carga.json
```

e executa o script:

```
sistema/back/docker/mongo/init/01-load-catmat.js
```

Se quiser recriar a carga:

```
cd sistema/back
docker compose down -v
docker compose up -d mongodb
```

2. Subir o Backend

Em outro terminal:

```
cd sistema/back
cp .env.example .env
npm install
npm run start:dev
```


No `.env` do backend, para Mongo local/Docker expondo `localhost:27017`, mantenha:

```
MONGO_URI=mongodb://localhost:27017
MONGO_DATABASE=patrimonio
MONGO_CATMAT_COLLECTION=catmat
```

Se o fluxo precisar da LLM, configure:

```
OPENAI_API_KEY=sua_chave
```

Backend disponível em:

```
http://localhost:3000
```

3. Subir o Frontend

Em outro terminal:

```
cd sistema/front
cp .env.example .env
npm install
npm run dev
```

No `.env` do frontend, aponte para a API:

```
VITE_API_BASE_URL=http://localhost:3000
```

Frontend disponível em:

```
http://localhost:5173
```

Teste Rápido

Com o sistema rodando, abra:

```
http://localhost:5173
```

Na tela inicial:

1. Baixe o modelo de planilha de entrada.
2. Envie um arquivo `.csv`, `.xlsx` ou `.xls`.
3. Confirme o envio.
4. Ao final, baixe o arquivo de saída e o arquivo de auditoria.

Também é possível testar a API diretamente:

```
curl -F "file=@caminho/para/planilha.xlsx" http://localhost:3000/catmat-classificacao/importar
```

Estrutura Relevante

```
sistema/
├─ docker-compose.yml
├─ .env.example
├─ back/
│   ├─ data/catmat/carga.json
│   ├─ docker/mongo/init/01-load-catmat.js
│   ├─ exports/
│   ├─ scripts/
│   ├─ src/
│   └─ test/fixtures/spreadsheets/teste.xlsx
└─ front/
    ├─ public/modelo-planilha-entrada.csv
    └─ src/
```

Observações

- Os arquivos gerados pela API ficam em `sistema/back/exports`.
- O volume `mongodb_data` preserva a base mesmo após `docker compose down`.
- Use `docker compose down -v` somente quando quiser apagar a base e refazer a carga inicial.
- O frontend chama a API usando `VITE_API_BASE_URL`; no Compose, o valor padrão é `http://localhost:3000`.